

5

진도 (공식 교재 기준)

- 아직 공식 강의자료에 나오지 않은 내용을 다룹니다.

강의자료

- 머신러닝 Lv.2
 1. AutoEncoder
 2. GAN
- 머신러닝 Lv.2 > 레포트
 1. 오토인코더 보고서
 2. GAN + Transformer 보고서

강의 주제

0차시: 이미지의 이해



이러한 이미지가 있다고 해 보자
컴퓨터에게 위 이미지는 숫자의 집합에 불과함
우선 각 색깔별로 아래와 같이 RGB 채널로 분리할 수 있다



그런 다음 각 채널의 각 픽셀의 밝기를 0~255로 나타낼 수 있을거임
숫자가 커질수록 해당 색 성분을 많이 포함함을 의미

만약 28x28의 아주 작은 컬러 이미지라면 총 숫자는 $28 \times 28 \times 3 = 2352$ 개의 숫자로 표현 가능함

근데 28x28 사이즈 이미지 누가 씀

요즘 기본이 1920x1080의 엄청난 고해상도이기 때문에, 이 숫자들을 모두 있는 그대로 다루기엔 뭔가 문제가 있을 수 있을거같음

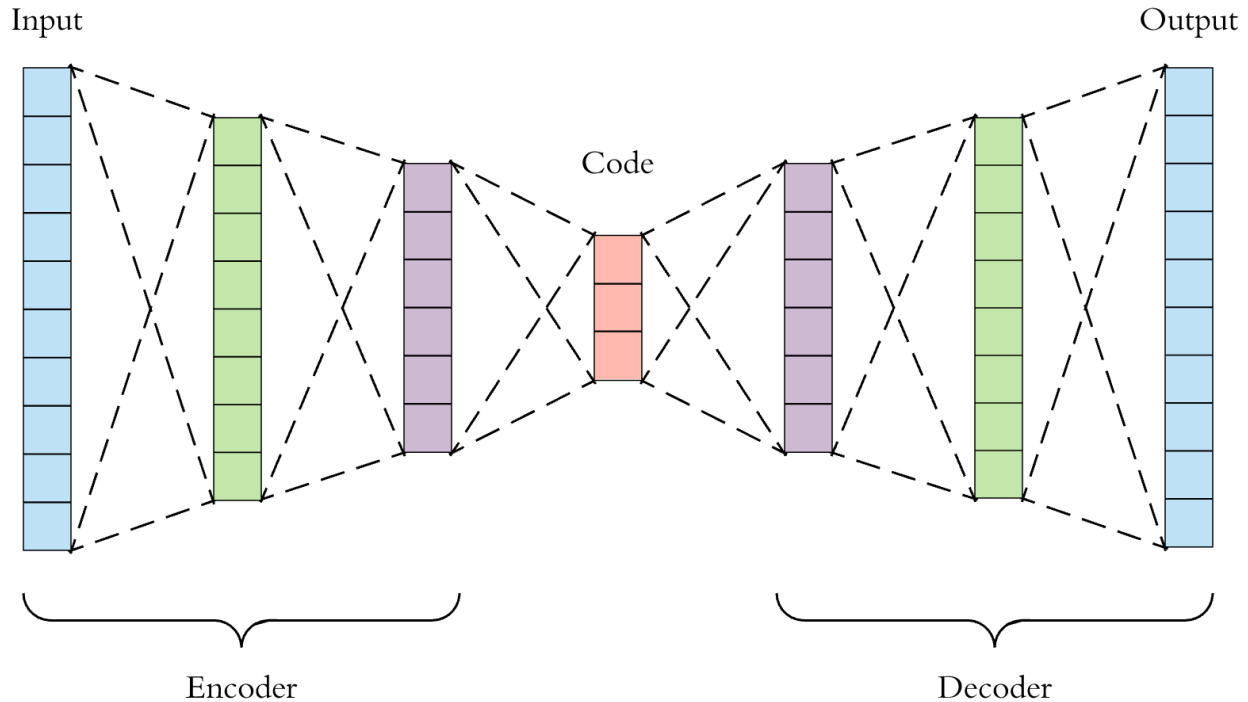
그럼 이미지의 핵심적인 특징만 압축해서 숫자 몇 개로 압축하면 다루기 수월하지 않을까?

1차시: AutoEncoder.pdf

배울 내용: AutoEncoder, Variational AutoEncoder

AutoEncoder

(여기서 Auto는 자동이 아니라 자기 자신을 의미함)



이건 자기 자신 인코더라는 뜻임

목표는 이미지를 압축해서 적은 숫자로 표현한 다음, 그걸 기반으로 원본 이미지를 복원하는거임

이게 가능하다면 1920x1080개의 픽셀들을 일일이 계산할 필요가 없겠지?

조금 더 자세하게는, 인코더는 압축을 하는 역할임

예를 들자면 위의 고양이 사진 $\rightarrow [0.72, -0.31, 1.24, 0.08, \dots]$ 의 벡터로 압축해주는 놈임

이렇게 압축된 공간을 Latent Space, 압축된 벡터를 Latent Vector라 부름

여기서 공간이란건 별게 아니고, 벡터가 그려지는 배경이라 생각하면 됨

[3] 이라는 1차원 벡터는 1차원 공간에 그려지고, [2, 6, 7] 이라는 3차원 벡터는 3차원 공간에 그려지는 식임

반면 디코더는 인코더가 압축한 Latent Vector를 기반으로 다시 원본 이미지를 구축하는 녀석임

(인코더랑 디코더 둘다 DNN 으로 구성됨)

오토인코더의 목표는 아주 단순함.

바로 "입력과 출력을 똑같이 하자" 임.

위의 고양이를 압축해서 만든 Latent Vector를 풀었더니 강아지 사진이 나오면 뭔가 잘못된 거겠지?

그래서 오토인코더의 loss는 두 이미지가 다를수록 커짐.

(loss에 대한 자세한 설명은 아래 VAE의 loss 부분 참고하셈)

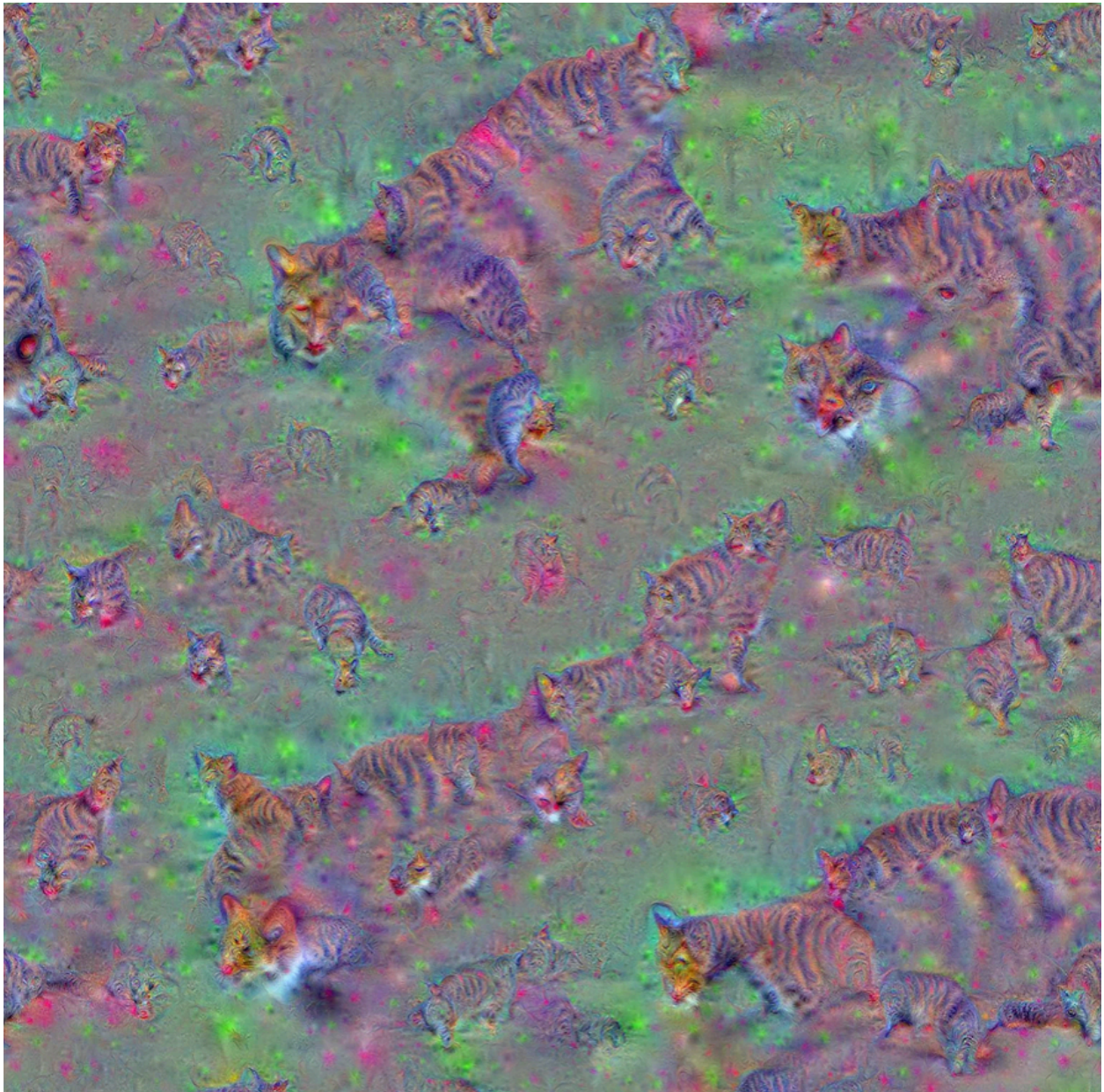
오토인코더의 문제

이제 이미지를 압축해서 편하게 다룰 수 있게 됨

그건 좋은데, 이제 새로운, 세상에 존재하지 않던 이미지를 만들어보고 싶은거임

그럼 아까 고양이 사진 $\rightarrow [0.72, -0.31, 1.24, 0.08, \dots]$ 이렇게 압축된 Latent Vector의 값을 조금만 수정하면 되지 않을까?

그래서 새로운 고양이 사진 $\rightarrow [0.74, -0.32, 1.25, 0.12, \dots]$ 이런 식으로 바꿔봄
그랬더니....



오토인코더가 약이라도 뻔 걸까?

똑똑한 사람들이 이 문제에 대해 고민해본 결과, 우리는 “인코더로부터 얻은 Latent Vector 주변에는 최소한 고양이랑 비슷한 이미지를 나타내는 Latent Vector가 있을 것이다” 라고 멋대로 단정한 것이 원인이었음

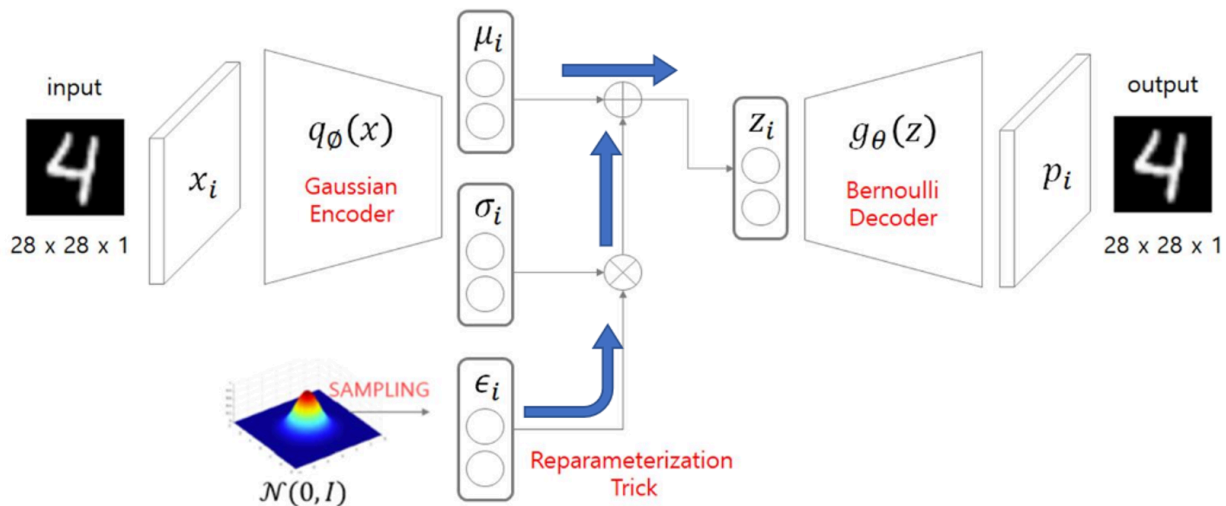
다시 말해, 오토인코더가 학습을 완료해서 고양이 사진의 벡터는 기가막히게 뽑는데, Latent Space에서 그 벡터 부분 외에는 뭐가 있을지 모르는 상태라는 거임

그래서 디코더가 한 번도 본 적 없는 이상한 벡터를 넣자 이런 이상한 이미지가 생긴거임 즉, 오토인코더는 압축과 복원은 잘 하지만 새로운 이미지를 생성하는데는 영 아닌 구조라는 결론이 나옴

그런데 생각해보면 Latent Space를 어떻게 규칙적으로 정렬만 하면 새로운 이미지를 뽑을 수 있을 것 같다 라는 생각이 들지 않나?

Variational AutoEncoder

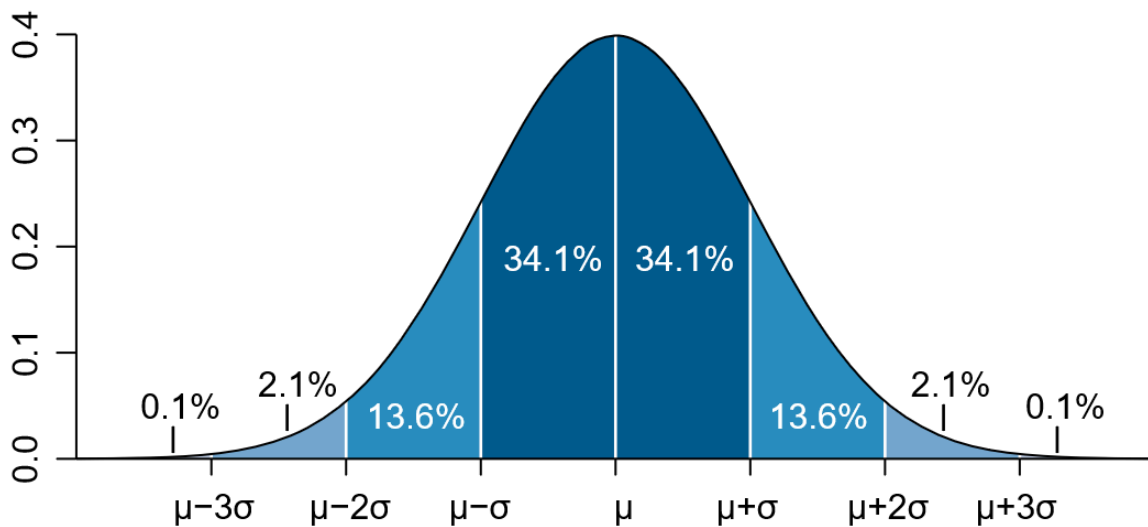
그래서 나온게 VAE다



아까 오토인코더보다는 뭔가 더 들어가서 복잡해졌지만, 큰 틀에서는 Input → Encoder → Latent Space(Latent Vector) → Decoder → Output 인건 같음

다만 결정적인 차이가 뭐냐면, 아까 오토인코더의 출력은 특정한 Latent Vector, 즉 **고양이 사진** → **[0.72, -0.31, 1.24, 0.08, ...]** 이런 특정한 벡터값이었음

반면 VAE같은 경우 출력으로 확률 분포를 생성하는데, 더 자세히 알아보자



VAE의 출력은 평균과 표준편차임

위 이미지는 평균이 μ , 표준편차가 σ 인 확률분포를 나타냄

→ 평균과 표준편차로 그려지는 확률분포를 앞으로 1차원 Gaussian Distribution 이라 할 거임

VAE 인코더의 출력이 μ, σ^2 이었다면, μ 에 가까이 위치한 벡터일수록 디코더로 입력될 확률이 커지는 구조임

VAE는 출력된 확률 분포를 기반으로 저기서 벡터를 하나 뽑아서, 디코더의 입력에 넣음
즉, 저 확률 분포에서 무작위 값을 하나 뽑는다고 생각하면 됨.

정리해보자

VAE 는 입력을 받아 인코더에서 압축하는것 까지는 오토인코더와 같음.

그러나 Latent Vector를 출력하는 오토인코더와 달리 애는 평균과 편차를 출력함 (사실상 확률 분포를 출력하는거임) 즉, 인코더의 출력은 곧 Gaussian Distribution임

이 Gaussian Distribution에서 z (Latent Vector 역할)를 하나 뽑음

그 다음 그걸 디코더에 넣어서 출력을 뽑는 구조임

왜 복잡하게 확률분포를 사용함?

그나마 쉽게 Latent Space를 정렬할 수 있는 방법이기 때문임

확률의 개념을 차용했기 때문에 Latent Vector들이 어느 정도 연속적이고 규칙적인 공간을 이루도록 학습됨 (이후에 배울 KLD가 이를 수행함)

그럼 고양이에 해당하는 Latent Vector와 강아지에 해당하는 Latent Vector 사이를 조금씩 이동하면

고양이 → 고양이 같은 강아지 → 강아지 같은 고양이 → 강아지 처럼 특징이 서서히 변할 수 있음

VAE의 핵심 문제와 해결책

디코더와 인코더는 DNN으로 이루어져 있음

그렇다면 이걸 역전파 해서 오차를 최소화하는 방향으로 학습을 진행하게 될텐데, 일단 역전파가 가능하기 위한 조건들을 잠시 살펴보자

1. loss가 존재할것
→ 아까 원본과 복원된 이미지의 차이를 loss로 하기로 했음
2. 업데이트될 weight가 존재할것
→ 인코더와 디코더는 DNN임. 당연히 weight 있음
3. 출력층 → 입력층이 모두 미분 가능한 구조일것
→ 인코더의 출력으로 나온 Gaussian Distribution에서 z 라는 값을 랜덤하게 하나 뽑아
⇒ 이 부분이 문제임. 랜덤값의 미분 =?

랜덤한 값의 미분은 들어본적도 없는 듯 하다

역전파가 가능하려면 이 랜덤값을 미분해야 하는데, 랜덤인데 어떻게 미분함

그래서 우리는 Trick을 쓸거다

정식 명칭은 Reparameterization Trick인데, 우리가 Latent Space에서 뽑는 랜덤값인 z 는 $z \sim N(\mu, \sigma^2)$ 라고 표현될 수 있음 (평균 μ , 표준편차 σ 인 확률분포에서 값 하나 뽑으라는 표기임)

여기서 확률분포에서 직접 z 를 랜덤하게 추출하는게 문제였음

왜 문제가 되냐면, 우리가 궁극적으로 필요한건 z 를 평균과 표준편차로 편미분한 결과값임

그런데 그냥 $z \sim N(\mu, \sigma^2)$ 라고만 하면 계산 과정 중간에 "확률분포에서 랜덤하게 하나 뽑기"라는 연산이 들어가고, 우리는 랜덤값의 미분은 어떻게 하는지 모른다

그럼 랜덤한 성질을 다른 곳으로 빼버리면 어떨까?

그렇다면 z 는 $z(\mu, \sigma)$ 형태의 미분가능한 함수로 나타내는게 좋을 듯 싶다. 문제는 z 가 랜덤하단 것이다

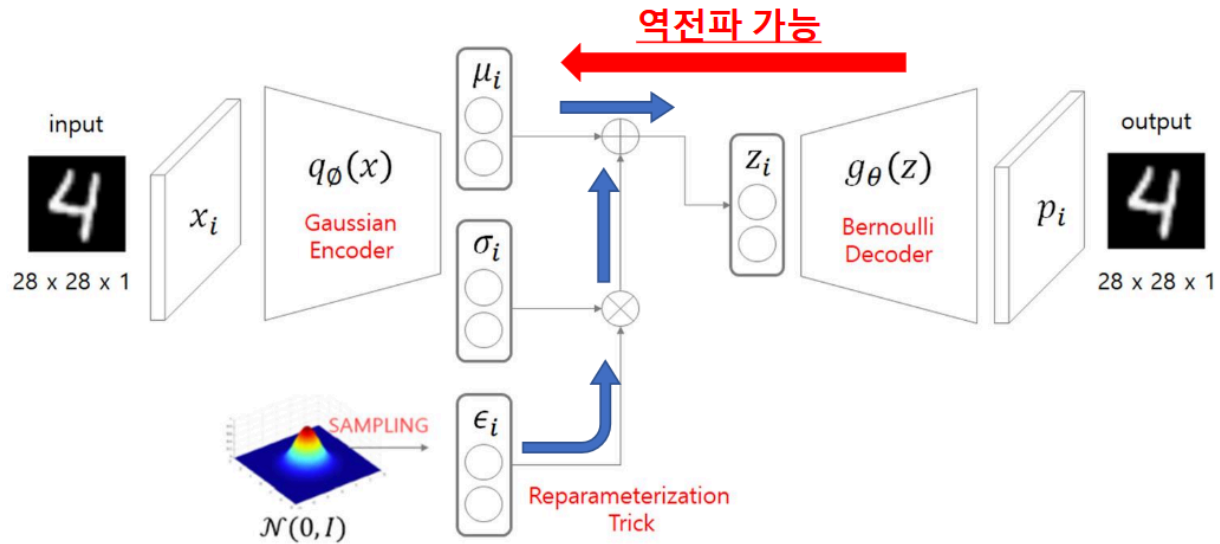
z 자체가 랜덤한 대신, 따로 랜덤한 값을 만들어 z 함수 안에 추가하면 어떨까?

$\epsilon \sim N(0, 1)$ 인 ϵ 을 뽑았다 가정하자

그걸로 z 를 정의해보면, $z = \mu + \epsilon\sigma$ 라고 쓰면 되지 않을까?

즉, 표준정규분포 $\epsilon \sim N(0, 1)$ 에서 ϵ 을 하나 뽑으면, 이 ϵ 은 쉽게 말해서 "평균에서 표준편차 몇 개만큼 떨어진 곳을 고를지"를 랜덤하게 정해주는 값으로 볼 수 있음.

이렇게 하면 편미분도 구할 수 있고, 개별 역전파 과정 중에는 ϵ 값을 고정시켜 놓으면 문제 없을 듯 하다



여기까지 인코더와 디코더의 구조, VAE의 Latent Space, Reparameterization Trick까지 알아봄

저기 디코더에 베르누이라고 쓰인건 디코더의 출력이 0.8 이면 그걸 "1일 확률 80%, 0일 확률 20%로 생각하겠다" 라는 뜻임

다음은 원본 이미지와 복원된 이미지 사이 loss를 어떻게 정의하는지 알아보도록 하자

VAE의 loss

VAE는 두 가지 종류의 작업을 동시에 잘 해내야 함

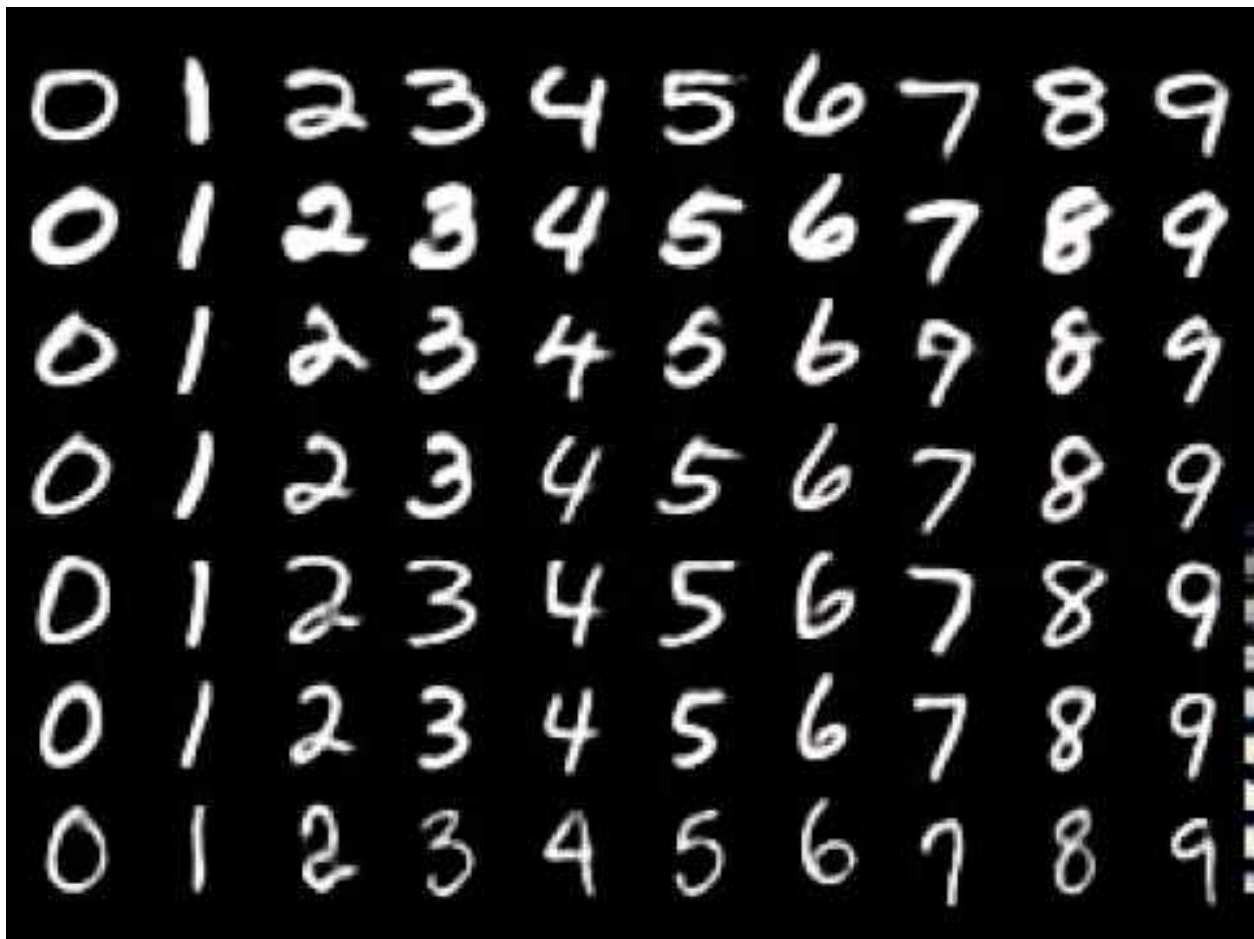
1. 원본 이미지 제대로 복원 $\Rightarrow$ Reconstruction Error
2. Latent Space가 우리가 원하는 확률분포와 비슷하게 정렬 $\Rightarrow$ KL Divergence

그래서 VAE의 loss = (Reconstruction Error) + (KL Divergence) 로 정의됨

이를 위해서 Reconstruction Error $\rightarrow$ KL Divergence의 개념 $\rightarrow$ KL Divergence $\rightarrow$ VAE Loss

순으로 알아보도록 하자

그 전에, 우리의 입력은 MNIST 데이터라고 하자



이렇게 생긴 손글씨 데이터인데, 각 숫자는 28x28x1 크기의 이미지이다.

밝기는 0~255 사이 값으로 표현됨

(아래의 공식들은 풀어보지 말고 그냥 그렇구나~ 하고 넘기셈)

1. Reconstruction Error

일단 Reconstruction Error의 공식부터 보고 가도록 하자

$$\mathbb{E}_{q_{\phi}(z|x_i)} [\log p(x_i|g_{\theta}(z))]$$

이 식을 우리의 입력인 MNIST에 맞춰 바꿔보면 아래와 같다

$$\sum_{j=1}^D [x_{i,j} \log p_{i,j} + (1 - x_{i,j}) \log(1 - p_{i,j})]$$

우선 표기부터 보도록 하자

i: 이 이미지는 몇 번째 이미지인가?

j: 그 이미지의 몇 번째 픽셀인가? (왼쪽 위가 맨 첫번째 픽셀)

p: 디코더가 복원한 그 픽셀의 밝기 값이 1일 확률은? (1이면 흰색, 0이면 검은색)

D: 784 (MNIST 의 전체 픽셀 개수에 대해 반복)

이다

그런데 식을 자세히 보다보면 이런 형태인 것을 알 수 있다

$$x \log(p) + (1 - x) \log(1 - p)$$

이거 Cross Entropy Error 식 아님?

맞고, 아까 위에서 디코더가 Bernoulli Decoder 이었기 때문에 Binary Cross Entropy, BCE라고 부름

(만약 Gaussian Decoder 였으면 MSE였음)

요약하자면, 디코더가 출력한 이미지의 각 픽셀에 대해 디코더가 예측한 픽셀이 1일 확률이 실제 픽셀값과 얼마나 들어맞는지 측정한다.

2. KL Divergence의 개념

이건 확률론적 개념인데, 차근차근 알아보자

만약 동전 1개를 던져서 아래와 같은 실험 결과가 나왔다고 가정해보자

H T H T H H T H T H H T H T H

이 실험 결과를 KLD로 해석해 볼 것이다

우선, KL Divergence는 두 확률분포가 얼마나 다른지를 재는 자와 같은 것임.

만약 동전 A: $P(H)=0.5$, $P(T)=0.5$ 이고,

동전 B: $P(H)=0.51$, $P(T)=0.49$ 이라면

두 분포는 거의 비슷하므로 KL Divergence가 작음.

반면 동전 C: $P(H)=0.99$, $P(T)=0.01$ 이라면

동전 C는 A와 굉장히 다르므로 KL Divergence가 큼.

즉, 분포가 비슷하다 → KL 작음, 분포가 다르다 → KL 은 크다.

우리의 목표는 VAE의 Latent Space가 우리가 원하는 확률분포와 비슷하게 정렬 이다.

확률분포가 비슷해야 하니 분포가 얼마나 비슷한지를 다루는 KL을 쓰면 되겠네?

3. KL Divergence

얘는 우선 정의부터 보고 갑시다

$$D_{KL}(q||p) = \int q(x) \log \frac{q(x)}{p(x)} dx$$

위 식은 q와 p라는 확률분포에 대해 이 두 확률분포 사이의 KL Divergence를 나타낸 식이다.

이게 정의임 (외울 필요는 없음 제발 외우지 마셈)

역시 식부터 보고 가자

$$KL(q_\phi(z|x_i)||p(z))$$

간단히 설명하자면 q_phi는 인코더를 의미하고, 이는 조건부 확률로 되어 있다
x_i는 입력이니 이는 입력을 인코더에 넣었을 때 z가 나온다는 말을 하고있는거임
그리고 뒤의 p(z)는 우리의 목표로, 표준정규분포 (N(0,1))을 의미한다
역시 이를 우리의 입력인 MNIST에 맞게 바꿔보자

$$\frac{1}{2} \sum_{j=1}^J (\mu_{i,j}^2 + \sigma_{i,j}^2 - \ln(\sigma_{i,j}^2) - 1)$$

식이 굉장히 복잡해보인다

사실 이걸 우리가 계산할 필요는 없는데, 위의 식은 인코더와 표준정규분포를 그냥 계산한 결과에 불과하기 때문이다

아까 위의 KLD 정의 식에 아래의 Gaussian Distribution 정의를 넣으면 위의 식이 나옴

$$Gaussian\ Distribution = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

4. VAE Loss

다시, 목표는

1. 입력 복원 : Reconstruction Error

2. Latent Space를 N(0,1) 근처로 정렬 : KL Divergence

이 두 수치는 모두 작을수록 좋다 $\Rightarrow$ RE + KL의 최소 찾으면 됨

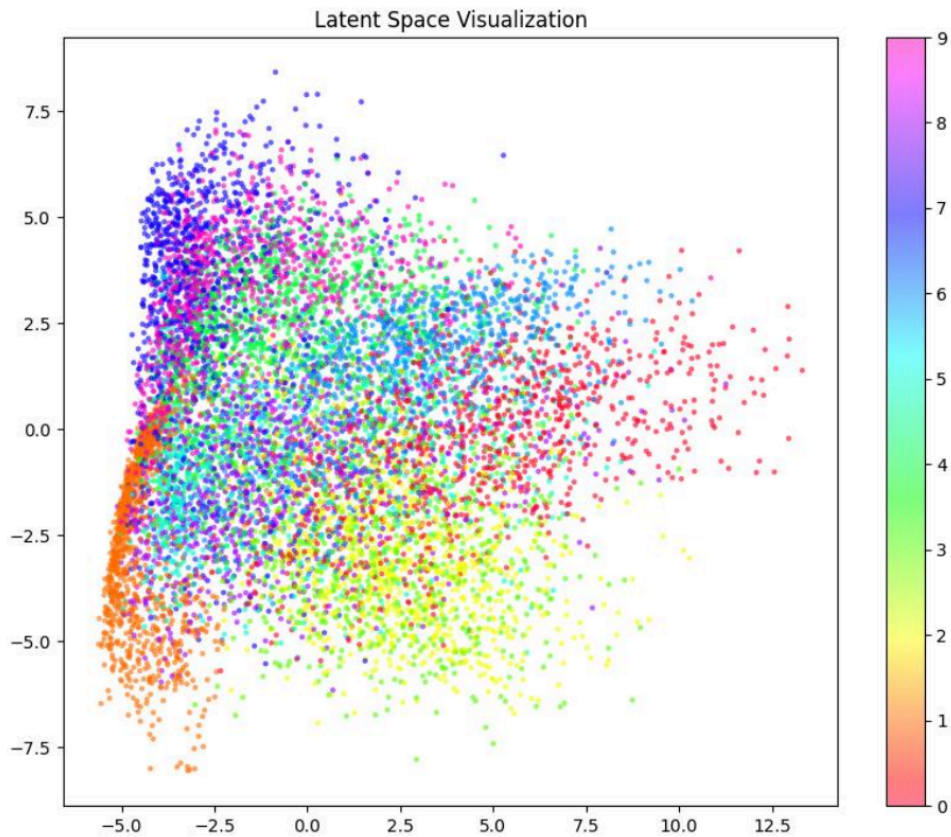
근데 문제는 Reconstruction Error는 디코더가 원본을 얼마나 높은 확률로 복원했나, 즉 Reconstruction Error가 높아야만 성능이 좋아짐
그러니 RE에다가 -1을 곱해주자
그럼 $-RE + KL$ 의 최소를 찾으면 될거같음

따라서 VAE 의 loss는 아래와 같이 정의된다

$$\arg \min_{\theta, \phi} \sum_i \left[-\mathbb{E}_{q_{\phi}(z|x_i)} [\log p(x_i|g_{\theta}(z))] + KL(q_{\phi}(z|x_i)||p(z)) \right]$$

즉 Reconstruction Error로 원본 이미지를 복원하면서도, KLD로 $N(0, I)$ 에서 너무 멀어지지 않도록 조정함으로서 Latent Space를 안정되고 정렬된 상태로 만듦
(고양이 벡터 바로 옆에 최소한 고양이랑 비슷한 벡터가 위치하도록)

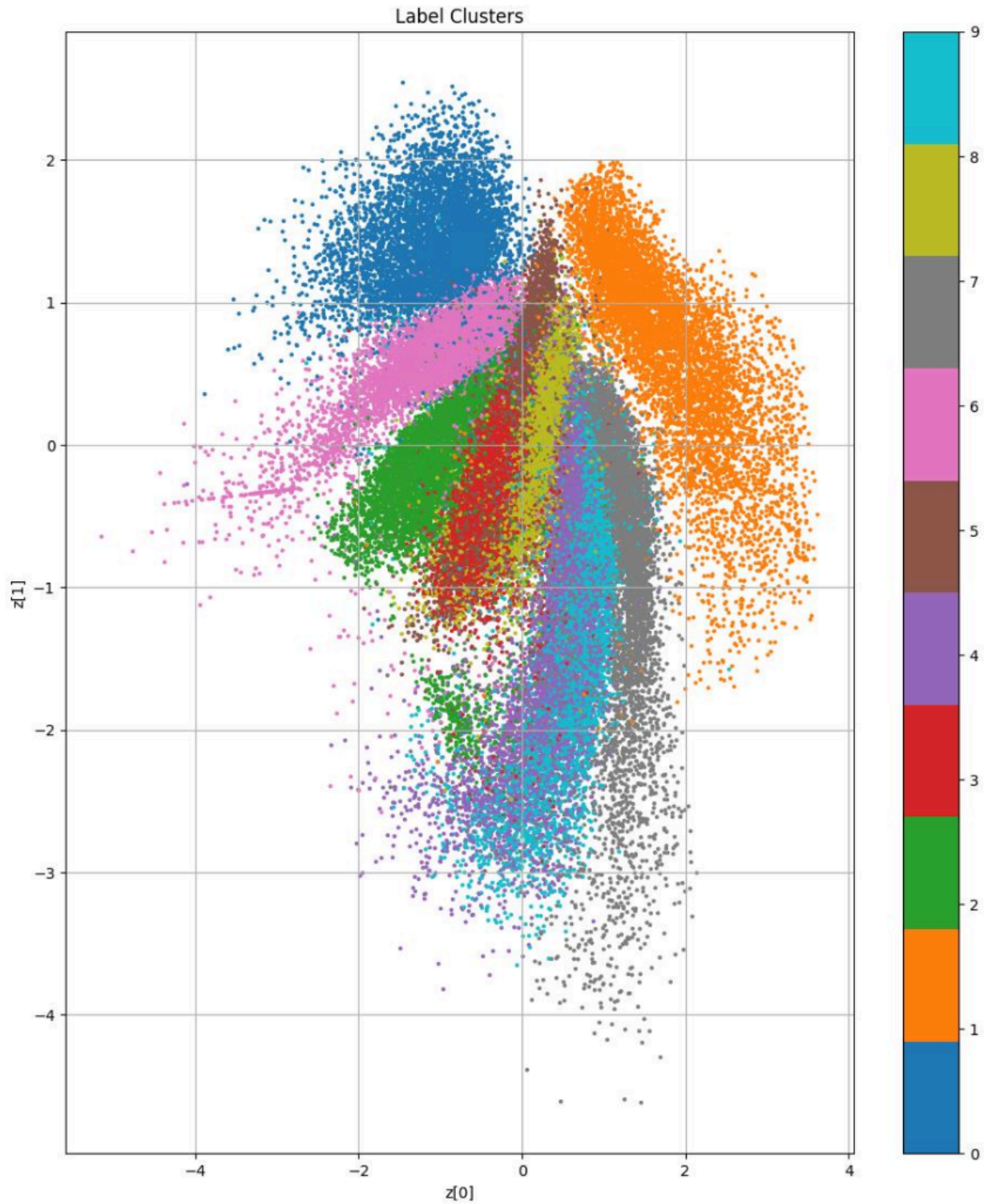
그걸 실제 결과로 봅시다



이건 내가 진행한 그냥 오토인코더의 결과임

MNIST 데이터 0~9 까지의 숫자를 Latent Vector로 압축해서 Latent Space에 압축해놓은 그림인데, 보면 알겠지만 각 색은 0~9 까지의 숫자를 뜻함

그런데 색이 이리저리 섞여 한 Latent Vector에서 조금만 옆으로 가도 숫자가 확 달라지는걸 볼 수 있음



이건 VAE 과제의 결과임
 이젠 각 색깔별로 어느정도 cluster를 이룬 것을 볼 수 있음

위의 일반 오토인코더에 비해 Latent Space가 정렬되었기 때문에 특정 Latent Vector 주변의 벡터들은 웬만하면 비슷한 숫자를 가리키게 됨

2차시: GAN.pdf

배울 내용: GAN

이번에 배울 내용은 GAN, Generative Adversarial Networks임
핵심 아이디어는 매우 간단함



여기는 인코더 디코더 대신 Generator(생성자)와 Discriminator(판별자)가 있다
생성자가 모조 데이터를 생성하면 판별자가 진짜 데이터와 가짜 데이터를 모두 보고 어느 것이
진짜 데이터인지 가려내는 시스템임
이걸 무한히 반복하면 생성자와 판별자가 서로 경쟁하면서 서로 성능이 올라감

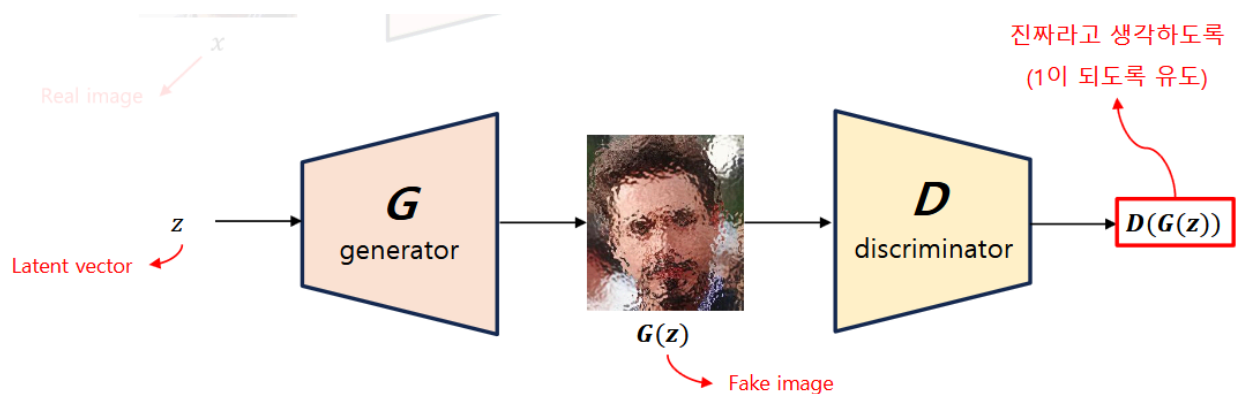


← 이거랑 완전히 똑같은 알고리즘임

비유를 하자면, 지폐 위조범 (생성자)은 경찰을 속이기 위해, 경찰은 실제 현금과 만들어진 위조지폐를 구분하기 위해 노력하는 구조임

이렇게 생성자와 판별자가 서로 적대적인 관계에서 경쟁하도록 학습 알고리즘이 짜여져 있어 GAN이라 부름

Generator

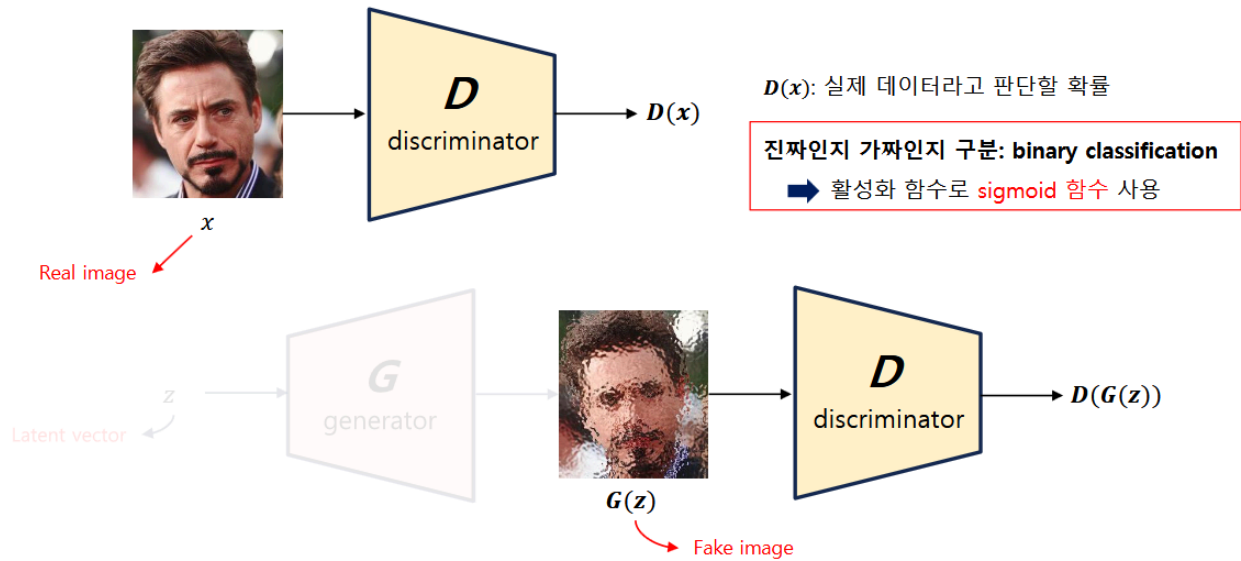


생성자는 랜덤한 노이즈를 받아 데이터를 생성함.

이렇게 생성된 데이터로 판별자를 속여서 판별자가 실제데이터라고 생각하도록 유도함

$D(x)$ 는 실제 데이터라고 판단할 확률임. 생성자의 목표는 $D(G(z))$ 가 1이 되도록 유도하는거임

Discriminator



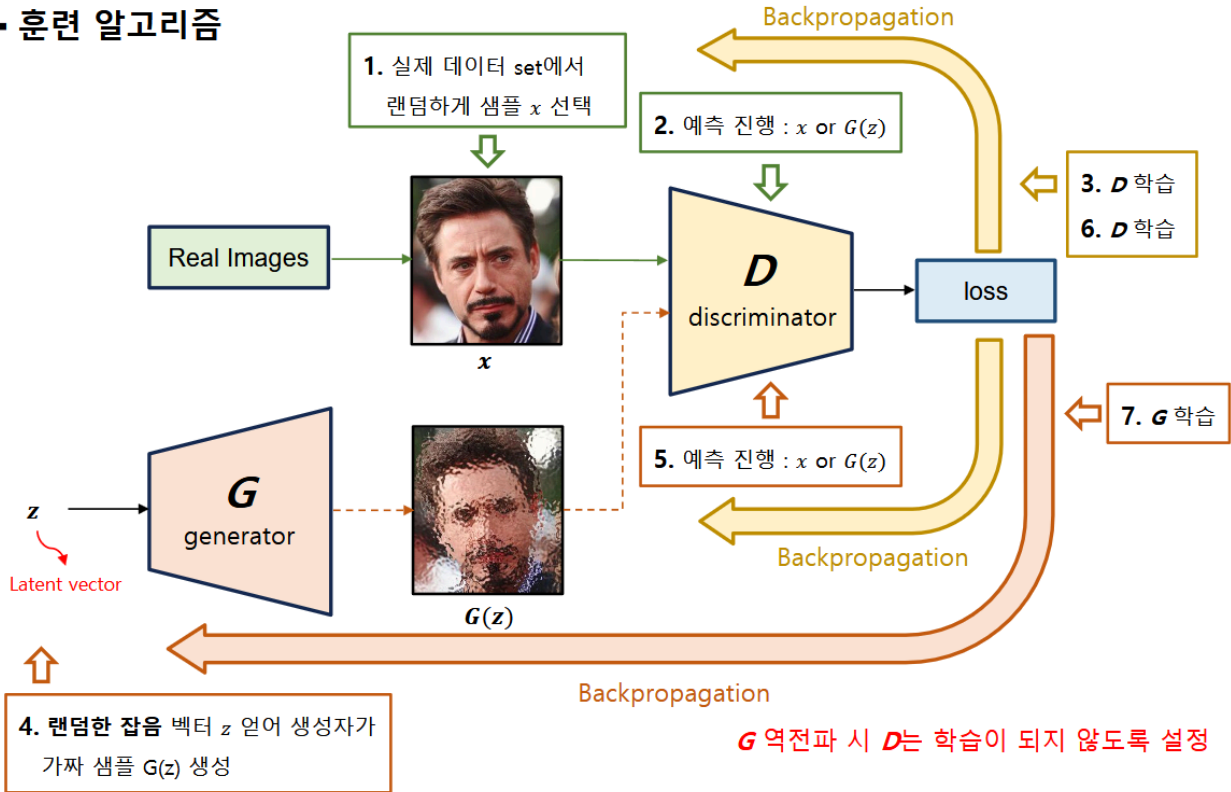
판별자는 출력 label이 있는 지도학습임.

실제 데이터가 들어가면(위) $D(x)$ 를 1로, 가짜 데이터가 들어가면(아래) $D(x)$ 를 0으로 구분하도록 함

클래스가 2개(True or False)이므로 Binary Classification 문제임

GAN의 역전파 및 학습

• 훈련 알고리즘



1. 실제 데이터셋에서 사진 하나 선택, 이후 그 이미지를 D에 넣음 (목표: $D(x) \approx 1$ 이 되도록)
2. D가 입력된 이미지 판별 후 오차를 계산해 역전파 수행
3. 역전파를 통해 D의 파라미터 업데이트 $\Rightarrow$ 진짜 사진을 진짜로 판별하도록 학습 완료
그런데 D는 진짜 뿐만 아니라 가짜 이미지도 알아봐야 함
4. G는 랜덤한 이미지를 만듦 (랜덤한 벡터를 입력받아 가짜 이미지 생성, 처음에는 이상한 노이즈 이미지가 나올 수 있음)
5. 가짜 사진인 $G(z)$ 를 D에 넣음
이 때 D는 이를 0으로 판단해야 하므로 목표는 $D(G(z)) \approx 0$ 임
위의 목표와 합치면 $D(x) \rightarrow 1, D(G(z)) \rightarrow 0$ 임
6. 이번에는 $D(G(z))$ 에 대해 0이 나오도록 역전파, D의 파라미터를 업데이트함 $\Rightarrow$ 가짜 사진을 가짜로 판별하도록 학습 완료
7. 이후 G까지 역전파하여 G의 파라미터를 업데이트하는데, 이때 D는 업데이트하지 않음
가짜 사진을 넣었으니 D의 출력은 0이 되어야 할 텐데, G의 목적은 D를 속이는 것이니 D의 출력이 1이 나와야 하기 때문
그래서 6번에서 D를 업데이트 한번 하고, 7번에서는 G를 업데이트하는데 이때 D의 파라미터는 고

정해놓음

다시 한번 보면, 7번 과정에서는 같은 가짜 이미지 $G(z)$ 를 보더라도 둘의 목표가 서로 다름.

D 입장: "가짜니까 0이라고 해야지": $D(G(z)) \rightarrow 0$

G 입장: "내 가짜를 진짜라고 속여야지": $D(G(z)) \rightarrow 1$

GAN의 Loss Function (Objective Function)

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

G는 이를 최소화하는 방향으로, D는 이를 최대화하는 방향으로 학습하게 됨
위에서 배운 내용을 식으로 쓴 것 뿐이라 설명은 생략함

GAN의 단점 및 해결책

GAN은 알아봤듯이 생성자와 판별자가 서로 싸우면서 서로 적대적으로 속고 속이는 방식으로 학습되는 구조임



이거 초기 GAN으로 생성한 가상 이미지인데, 이전의 VAE같은 것만 쓰다가 GAN을 쓰니 성능이 비약적으로 발전함

그러나 G와 D가 경쟁하면서 학습하는 근본적인 특성상 학습이 굉장히 불안정하고 까다로움
게다가 G가 D를 속이는데 목표이다보니 D를 잘 속일 수 있는 특정한 이미지만 계속 만드는 문제도 있었음 (Mode Collapse라 부름)

또한 D가 너무 강력해져서 G가 D를 속일 엄두도 내지 못할 수 있고, 반대로 G가 D를 너무 잘 속일 수도 있음

G와 D를 만들어서 이러한 문제가 생긴다면, G와 D를 없애면 되지 않을까?

→ Diffusion Model